

1. Consider that you are owning a supermarket mall and through membership cards, you have some basic data about your customers like Customer ID, age, gender, annual income and spending score. For the above scenario, the Problem Statement was You want to understand the customers who can easily converge [Target Customers] so that the data can be given to the marketing team and plan the strategy accordingly. For the above scenario prepare a dataset and perform **Clustering Analysis** to segment the customers in the Mall. There are clearly Five segments of Customers based on their Annual Income and Spending Score namely *Usual Customers*, *Priority Customers*, *Senior Citizen Target Customers*, and *Young Target Customers*. Sample data

|   | CustomerID | Gender | Age | Annual Income (k\$) | Spending Score (1-100) |
|---|------------|--------|-----|---------------------|------------------------|
| 0 | 1          | Male   | 19  | 15                  | 39                     |
| 1 | 2          | Male   | 21  | 15                  | 81                     |
| 2 | 3          | Female | 20  | 16                  | 6                      |
| 3 | 4          | Female | 23  | 16                  | 77                     |
| 4 | 5          | Female | 31  | 17                  | 40                     |

**Weka Explorer**

Preprocess Classify Cluster Associate Select attributes Visualize

Clusterer

Choose **SimpleKMeans** -init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 5 -A "weka.core.EuclideanDistance" -R first-last -I 500 -num-slots 1 -S 10

Cluster mode

☒ Use training set

☐ Supplied test set Set...

☐ Percentage split % 66

☐ Classes to clusters evaluation (Num) SpendingScore

☒ Store clusters for visualization

Ignore attributes

Start Stop

Result list (right-click for options)

09:40:12 - SimpleKMeans

Cluster output

```

=== Run information ===

Scheme: weka.clusterers.SimpleKMeans -init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 5 -A "weka.core.EuclideanDistance" -R first-last -I 500 -num-slots 1 -S 10
Relation: customer
Instances: 15
Attributes: 5
  CustomerID
  Gender
  Age
  AnnualIncome
  SpendingScore
Test mode: evaluate on training data

=== Clustering model (full training set) ===

KMeans
=====

Number of iterations: 3
Within cluster sum of squared errors: 2.12251116809571

Initial starting points (random):

Cluster 0: 4, Female, 23, 16000, 77
Cluster 1: 5, Male, 64, 19000, 3
Cluster 2: 12, Female, 35, 20000, 99
Cluster 3: 7, Female, 35, 18000, 6
Cluster 4: 8, Female, 23, 18000, 94

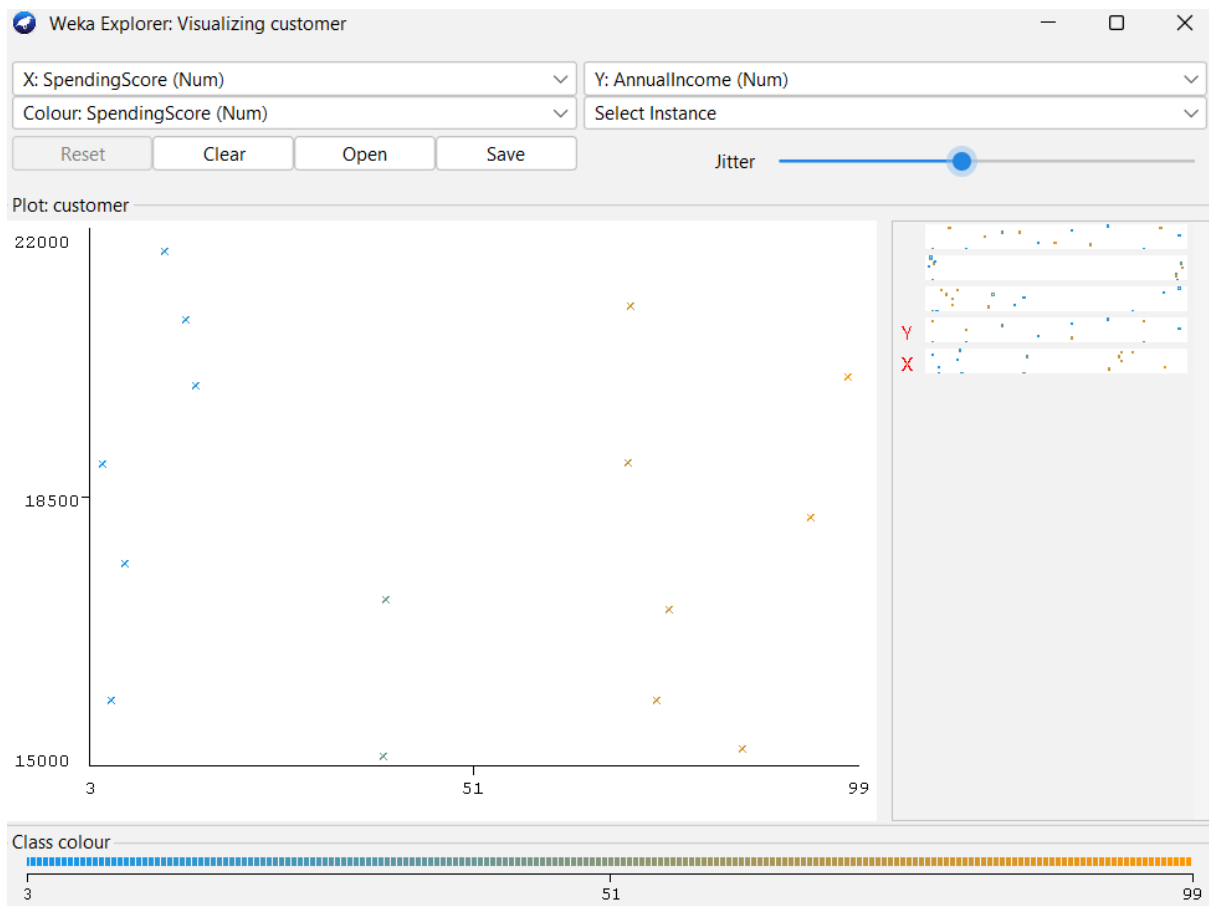
Missing values globally replaced with mean/mode

Final cluster centroids:

Attribute      Full Data      Cluster#
              (15.0)        (3.0)        (3.0)        (1.0)        (4.0)        (4.0)
=====
CustomerID      8              3      11.6667      14              7              8.5
Gender          Female         Male      Male      Male      Female      Female
Age             33.9333       20.6667   56         24         36         27.75
AnnualIncome    18266.6667    15666.6667 20333.3333 21000      18000      18250
SpendingScore   42.6667       26.3333   40         77         16.75     26.6
  
```

Status OK

Log



2. Create the following dataset using CSV file format. To perform cluster analysis using K-Means in WEKA. To change the cluster size and plot the graph and illustrate the visualization of cluster.

| EmployeeID | Gender | Age | Salary | Credit |
|------------|--------|-----|--------|--------|
| 111        | Male   | 28  | 150000 | 39     |
| 222        | Male   | 25  | 150000 | 27     |
| 333        | Female | 26  | 160000 | 42     |
| 444        | Female | 25  | 160000 | 40     |
| 555        | Female | 30  | 170000 | 64     |
| 666        | Male   | 29  | 200000 | 72     |

**Weka Explorer**

Preprocess Classify **Cluster** Associate Select attributes Visualize

Clusterer  
Choose **SimpleKMeans** -init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 2 -A "weka.core.EuclideanDistance -R first-last" -I 500 -num-slots 1 -S 10

Cluster mode  
☒ Use training set  
☐ Supplied test set Set...  
☐ Percentage split % 66  
☐ Classes to clusters evaluation (Num) Credit  
☒ Store clusters for visualization

Ignore attributes  
Start Stop

Result list (right-click for options)  
 09:40:12 - SimpleKMeans  
 09:57:01 - SimpleKMeans

Cluster output  
 test mode: evaluate on training data  
 === Clustering model (full training set) ===  
 kMeans  
 =====  
 Number of iterations: 2  
 Within cluster sum of squared errors: 2.9514403292181073  
 Initial starting points (random):  
 Cluster 0: 444,Female,25,160000,40  
 Cluster 1: 111,Male,28,150000,39  
 Missing values globally replaced with mean/mode  
 Final cluster centroids:  

| Attribute  | Full Data | Cluster#    | 0           | 1     |
|------------|-----------|-------------|-------------|-------|
|            | (6.0)     |             | (3.0)       | (3.0) |
| EmployeeID | 388.5     | 444         | 333         |       |
| Gender     | Male      | Female      | Male        |       |
| Age        | 27.1667   | 27          | 27.3333     |       |
| Salary     | 165000    | 163333.3333 | 166666.6667 |       |
| Credit     | 47.3333   | 46.6667     | 46          |       |

 Time taken to build model (full training data) : 0 seconds  
 === Model and evaluation on training set ===  
 Clustered Instances  
 0 3 ( 50%)  
 1 3 ( 50%)

Status  
OK

**Weka Explorer**

Preprocess Classify **Cluster** Associate Select attributes Visualize

Clusterer  
Choose **SimpleKMeans** -init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 3 -A "weka.core.EuclideanDistance -R first-last" -I 500 -num-slots 1 -S 10

Cluster mode  
☒ Use training set  
☐ Supplied test set Set...  
☐ Percentage split % 66  
☐ Classes to clusters evaluation (Num) Credit  
☒ Store clusters for visualization

Ignore attributes  
Start Stop

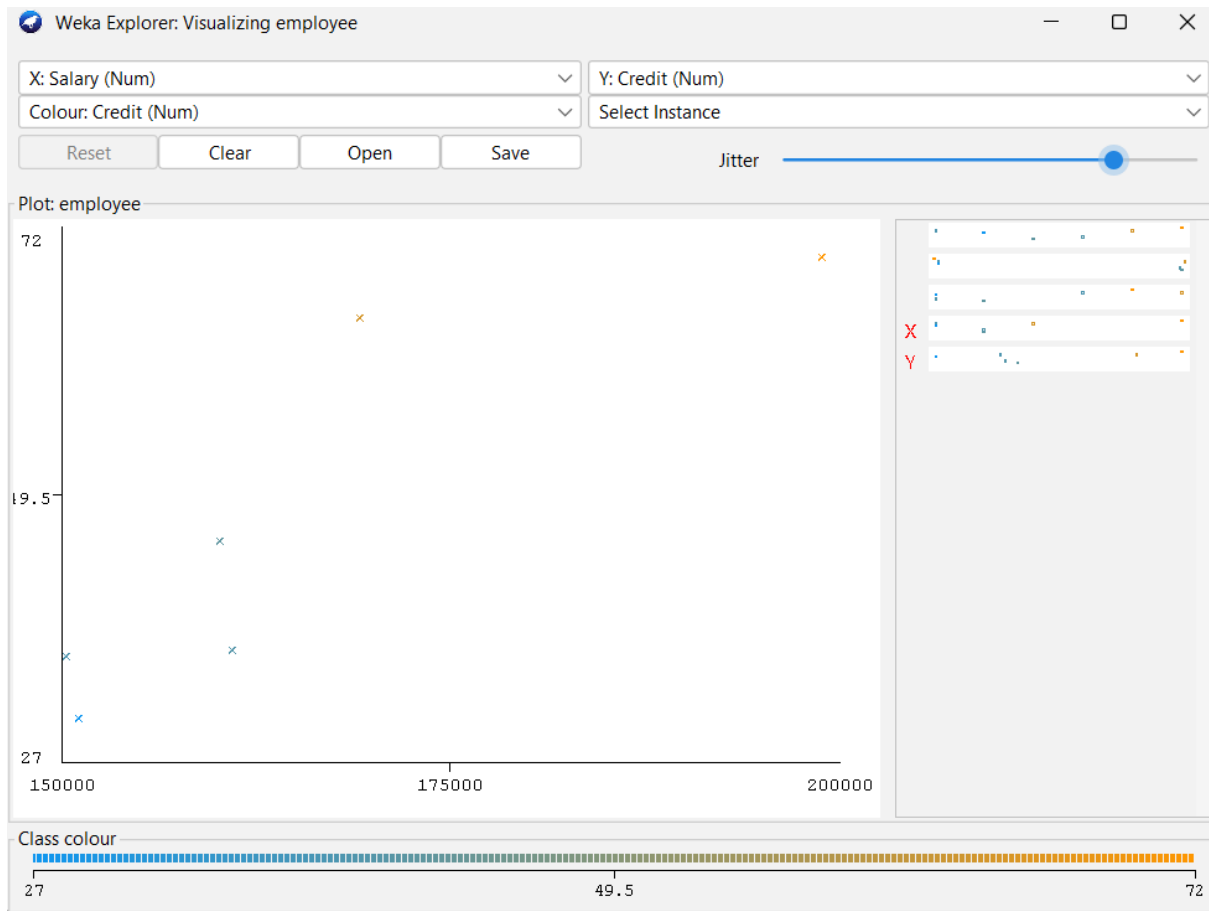
Result list (right-click for options)  
 09:40:12 - SimpleKMeans  
 09:57:01 - SimpleKMeans  
 09:57:50 - SimpleKMeans

Cluster output  
 === Clustering model (full training set) ===  
 kMeans  
 =====  
 Number of iterations: 3  
 Within cluster sum of squared errors: 1.077366255144033  
 Initial starting points (random):  
 Cluster 0: 444,Female,25,160000,40  
 Cluster 1: 111,Male,28,150000,39  
 Cluster 2: 222,Male,25,150000,27  
 Missing values globally replaced with mean/mode  
 Final cluster centroids:  

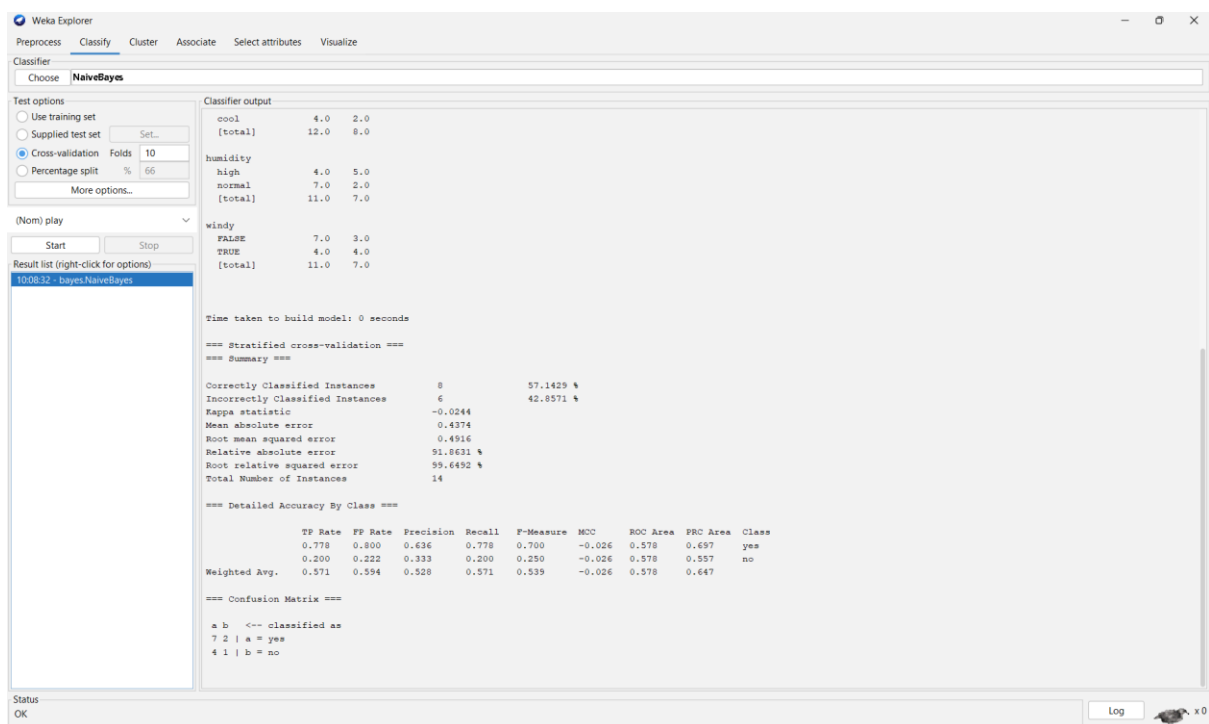
| Attribute  | Full Data | Cluster#    | 0      | 1      | 2     |
|------------|-----------|-------------|--------|--------|-------|
|            | (6.0)     |             | (3.0)  | (1.0)  | (2.0) |
| EmployeeID | 388.5     | 444         | 444    | 166.5  |       |
| Gender     | Male      | Female      | Male   | Male   |       |
| Age        | 27.1667   | 27          | 29     | 26.5   |       |
| Salary     | 165000    | 163333.3333 | 200000 | 150000 |       |
| Credit     | 47.3333   | 46.6667     | 72     | 33     |       |

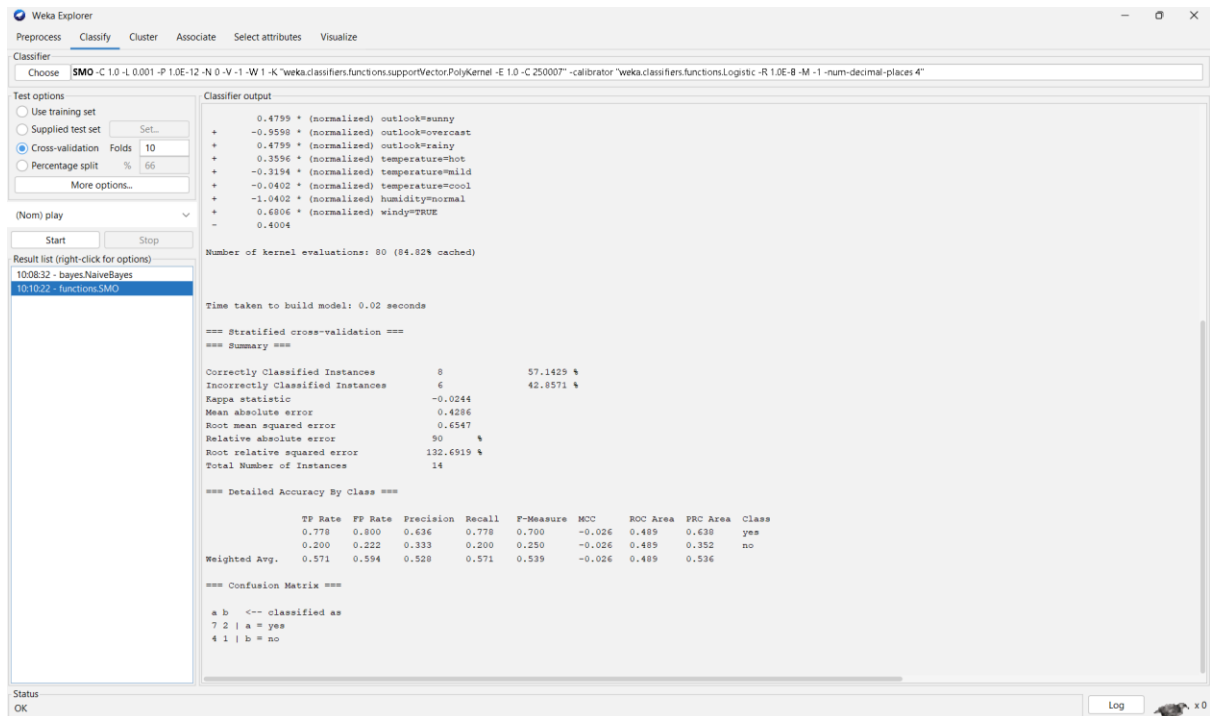
 Time taken to build model (full training data) : 0 seconds  
 === Model and evaluation on training set ===  
 Clustered Instances  
 0 3 ( 50%)  
 1 1 ( 17%)  
 2 2 ( 33%)

Status  
OK



3. Prediction of categorical data using Naïve Bayes classification through WEKA using any datasets. Compare the Naïve Bayes algorithm with SVM using the summary of results given by the classifiers and plot the graph.





4.The following list of persons with vegetarian or not details given in the table. How will you find out how many of them are vegetarian and how many of them are non-vegetarian? Which type of the person total count is greater value?

| Person     | Gopu | Babu | Baby | Gopal | Krishna | Jai | Dev | Malini | Hema | Anu |
|------------|------|------|------|-------|---------|-----|-----|--------|------|-----|
| Vegetarian | yes  | yes  | yes  | no    | yes     | no  | no  | yes    | yes  | yes |

| Person  | Vegetarian |
|---------|------------|
| Gopu    | Yes        |
| Babu    | Yes        |
| Baby    | Yes        |
| Gopal   | No         |
| Krishna | Yes        |
| Jai     | No         |
| Dev     | No         |

| Person | Vegetarian |
|--------|------------|
| Malini | Yes        |
| Hema   | Yes        |
| Anu    | Yes        |

**Step 1: Count Yes**

Yes values:

**Gopu, Babu, Baby, Krishna, Malini, Hema, Anu**

So:

**Vegetarian = 7**

|              |   |    |   |    |    |    |   |     |    |     |     |           |
|--------------|---|----|---|----|----|----|---|-----|----|-----|-----|-----------|
| <b>Step</b>  |   |    |   |    |    |    |   |     |    |     |     | <b>2:</b> |
| <b>Count</b> | x | 4  | 1 | 5  | 7  | 10 | 2 | 50  | 25 | 90  | 36  | <b>No</b> |
| No           | y | 12 | 5 | 13 | 19 | 31 | 7 | 153 | 72 | 275 | 110 |           |

values:

**Gopal, Jai, Dev**

So:

**Non-Vegetarian = 3**

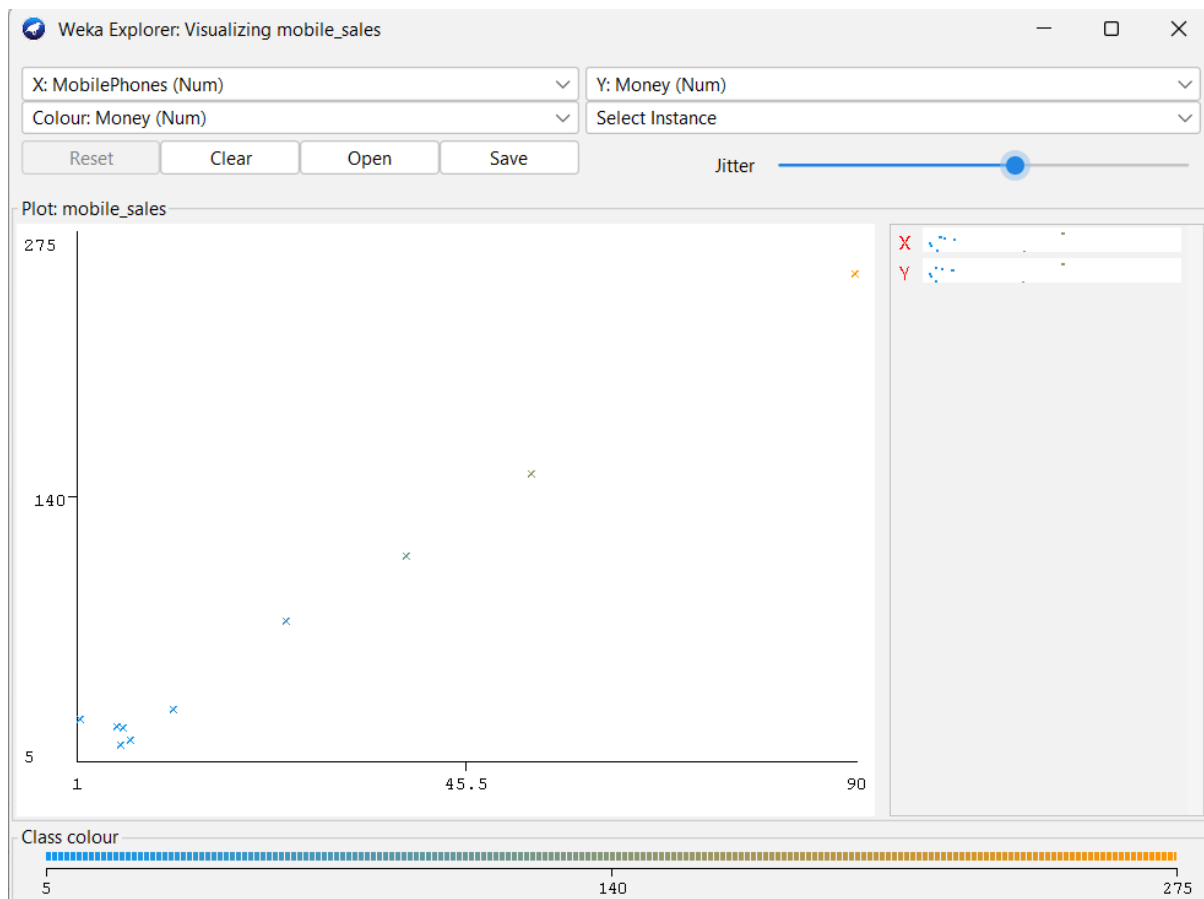
**Final Answer**

Vegetarian = 7

Non-Vegetarian = 3

Therefore, **the number of vegetarians is greater.**

5.The following table would be plotted as (x,y) points, with the first column being the x values as number of mobile phones sold and the second column being the y values as money. To use the scatter plot for how many mobile phones sold.



6. Generate rules using FP growth algorithm using the given dataset which has the following transactions with items purchased: Consider the values as support=50% and confidence=75%.

| Transaction ID | Items Purchased           |
|----------------|---------------------------|
| 1              | Bread, Cheese, Egg, Juice |
| 2              | Bread, Cheese, Juice      |
| 3              | Bread, Milk, Yogurt       |
| 4              | Bread, Juice, Milk        |
| 5              | Cheese, Juice, Milk       |

Weka Explorer

Preprocess   Classify   Cluster   **Associate**   Select attributes   Visualize

Associator

Choose **FPGrowth** -P 2 -I -1 -N 10 -T 0 -C 0.9 -D 0.05 -U 1.0 -M 0.1

Start   Stop

Result list (right-click for ...)

10:25:22 - FPGrowth

Associator output

```
=== Run information ===

Scheme:      weka.associations.FPGrowth -P 2 -I -1 -N 10 -T 0 -C 0.9 -D 0.05 -U 1.0 -M 0.1
Relation:    transactions
Instances:   5
Attributes:  5
             Bread
             Milk
             Butter
             Eggs
             Juice

=== Associator model (full training set) ===

FPGrowth found 2 rules (displaying top 2)

1. [Milk=no]: 1 ==> [Juice=no]: 1 <conf:(1)> lift:(2.5) lev:(0.12) conv:(0.6)
2. [Bread=no]: 1 ==> [Juice=no]: 1 <conf:(1)> lift:(2.5) lev:(0.12) conv:(0.6)
```

Weka Explorer

Preprocess   Classify   Cluster   **Associate**   Select attributes   Visualize

Associator

Choose **FPGrowth** -P 2 -I -1 -N 10 -T 0 -C 0.9 -D 0.05 -U 1.0 -M 0.5

Start   Stop

Result list (right-click for ...)

10:25:22 - FPGrowth  
10:34:23 - FPGrowth

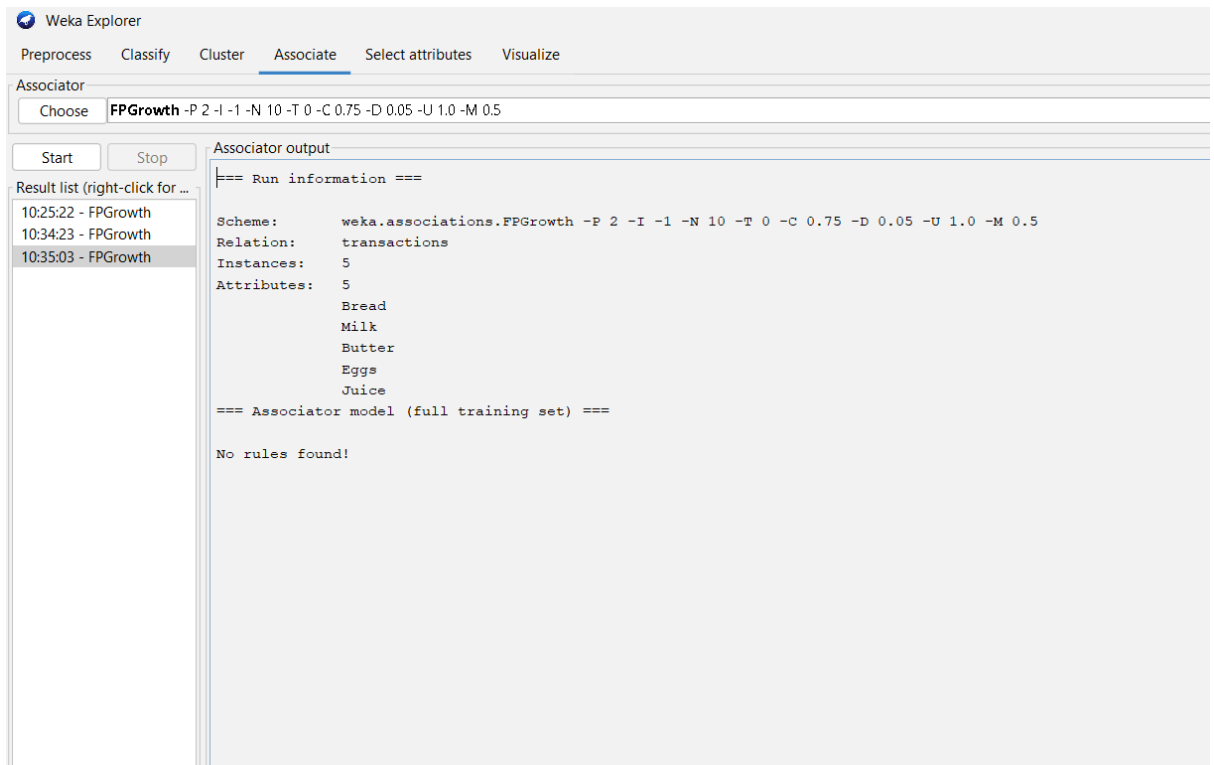
Associator output

```
=== Run information ===

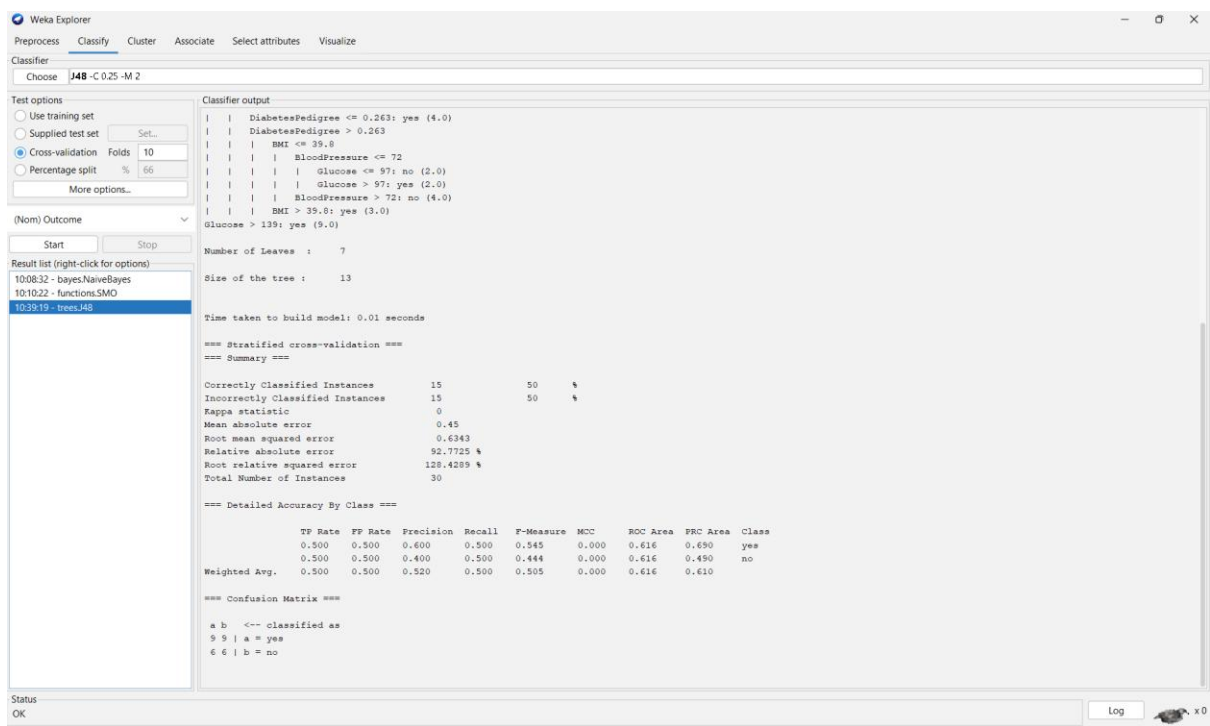
Scheme:      weka.associations.FPGrowth -P 2 -I -1 -N 10 -T 0 -C 0.9 -D 0.05 -U 1.0 -M 0.5
Relation:    transactions
Instances:   5
Attributes:  5
             Bread
             Milk
             Butter
             Eggs
             Juice

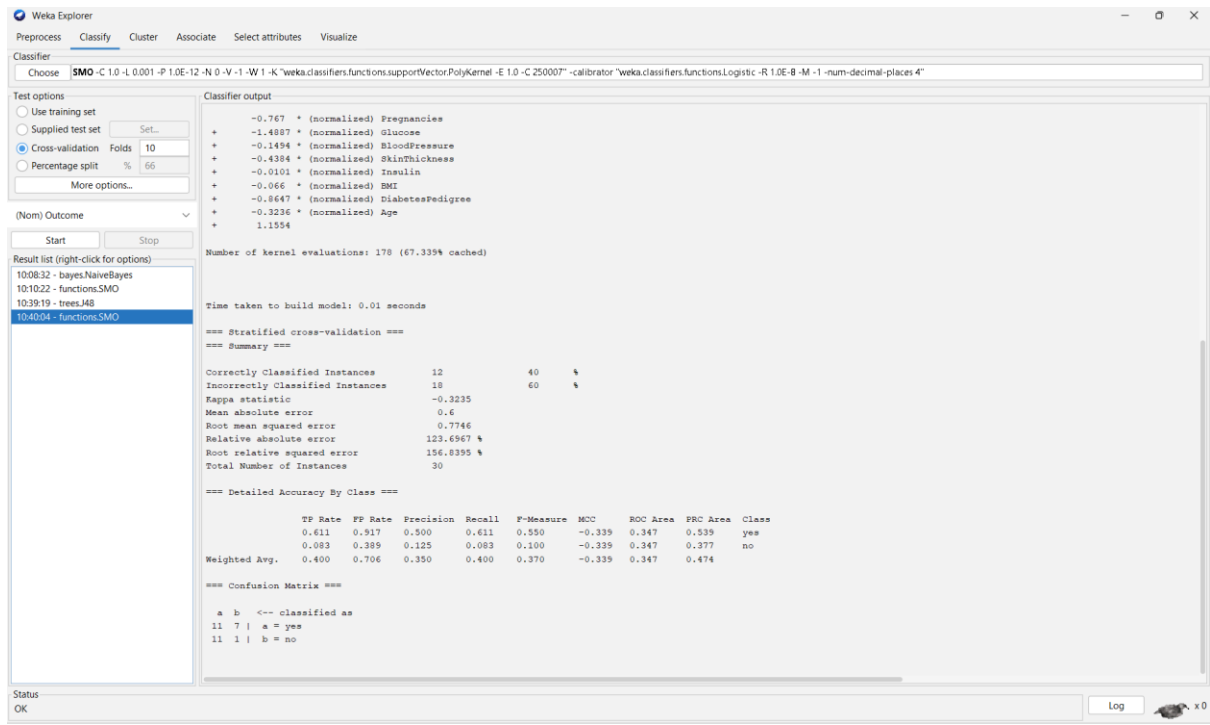
=== Associator model (full training set) ===

No rules found!
```



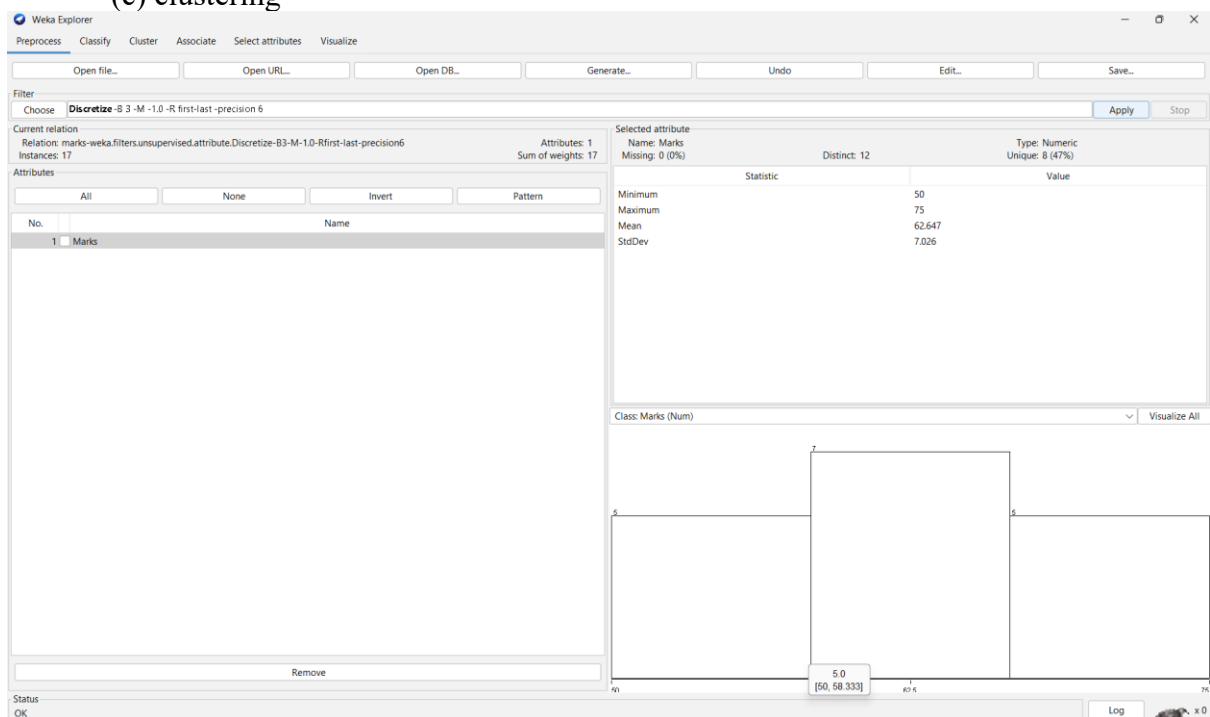
**7. Prediction of Diabetes Data using Decision tree classifier in WEKA. Compare it with Support Vector Machine classifier. Show the result accuracy and F1 measure calculation .Plot the graph and explain the summary of results.**

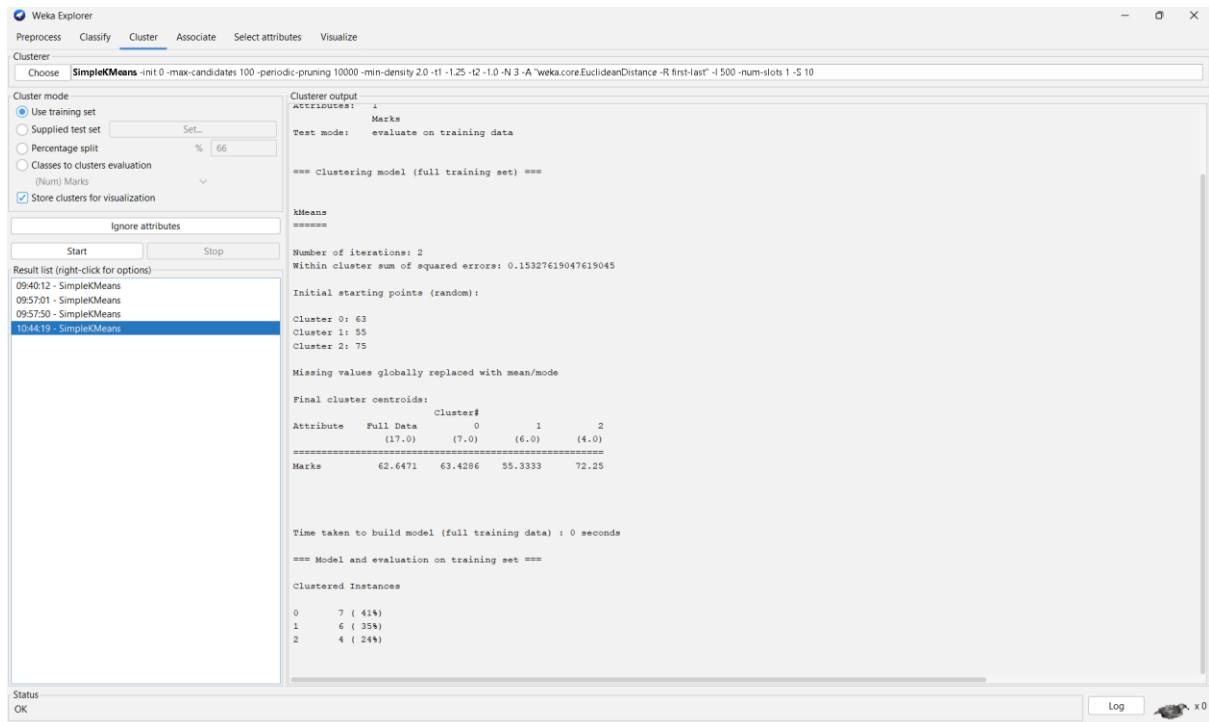




8. Implement of the R script using marks scored by a student in his model exam has been sorted as follows: 55, 60, 71, 63, 55, 65, 50, 55, 58, 59, 61, 63, 65, 67, 71, 72, 75. Partition them into three bins by each of the following methods. Plot the data points using histogram.

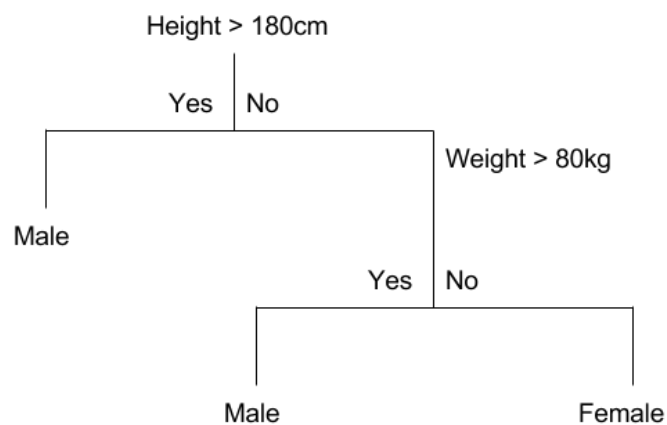
- equal-frequency (equi-depth) partitioning
- equal-width partitioning
- clustering





9. Consider this Decision tree :

- create the data set for the below tree using ARFF format and calculate accuracy and decision for the same
- Using this decision tree generate the rules based on rule based induction.
- Compare both the algorithms and plot the confusion matrix.



**Weka Explorer**

Preprocess Classify Cluster Associate Select attributes Visualize

Classifier: Choose **J48 - C 0.25 - M 2**

Test options:  
☐ Use training set  
☐ Supplied test set  
☒ Cross-validation Folds: **10**  
☐ Percentage split % **66**  
 More options...

(Nom) Gender

Start Stop

Result list (right-click for options):  
 100832 - bayes.NaiveBayes  
 101022 - functions.SMO  
 103919 - trees.J48  
 104004 - functions.SMO  
**104728 - trees.J48**

Classifier output:

```

=== Classifier model (full training set) ===
J48 pruned tree
-----
Weight <= 75
| Height <= 178: Female (4.0)
| Height > 178: Male (2.0)
Weight > 75: Male (4.0)

Number of Leaves : 3
Size of the tree : 5

Time taken to build model: 0 seconds

=== Stratified cross-validation ===
=== Summary ===
Correctly Classified Instances      5      50 %
Incorrectly Classified Instances    5      50 %
Kappa statistic                    -0.087
Mean absolute error                 0.4314
Root mean squared error             0.6123
Relative absolute error             81.8227 %
Root relative squared error        114.5077 %
Total Number of Instances         10

=== Detailed Accuracy By Class ===
          TP Rate  FP Rate  Precision  Recall  F-Measure  MCC  ROC Area  PRC Area  Class
Weighted Avg.   0.500   0.583   0.476   0.500   0.484   -0.089   0.667   0.713   Female

=== Confusion Matrix ===
a b  <-- classified as
4 2 | a = Male
3 1 | b = Female
  
```

Status: OK

---

**Weka Explorer**

Preprocess Classify Cluster Associate Select attributes Visualize

Classifier: Choose **JRip - F 3 - N 2.0 - O 2 - S 1**

Test options:  
☐ Use training set  
☐ Supplied test set  
☒ Cross-validation Folds: **10**  
☐ Percentage split % **66**  
 More options...

(Nom) Gender

Start Stop

Result list (right-click for options):  
 100832 - bayes.NaiveBayes  
 101022 - functions.SMO  
 103919 - trees.J48  
 104004 - functions.SMO  
 104728 - trees.J48  
**104814 - rules.Jrip**

Classifier output:

```

Weight
Gender
Test mode: 10-fold cross-validation

=== Classifier model (full training set) ===
JRip rules:
=====
(Weight <= 75) and (Weight <= 178) => Gender=Female (4.0/0.0)
=> Gender=Male (6.0/0.0)

Number of Rules : 2

Time taken to build model: 0 seconds

=== Stratified cross-validation ===
=== Summary ===
Correctly Classified Instances      7      70 %
Incorrectly Classified Instances    3      30 %
Kappa statistic                    0.3478
Mean absolute error                 0.2905
Root mean squared error             0.453
Relative absolute error             55.0903 %
Root relative squared error        84.7149 %
Total Number of Instances         10

=== Detailed Accuracy By Class ===
          TP Rate  FP Rate  Precision  Recall  F-Measure  MCC  ROC Area  PRC Area  Class
Weighted Avg.   0.700   0.367   0.695   0.700   0.690   0.356   0.771   0.782   Female

=== Confusion Matrix ===
a b  <-- classified as
5 1 | a = Male
2 2 | b = Female
  
```

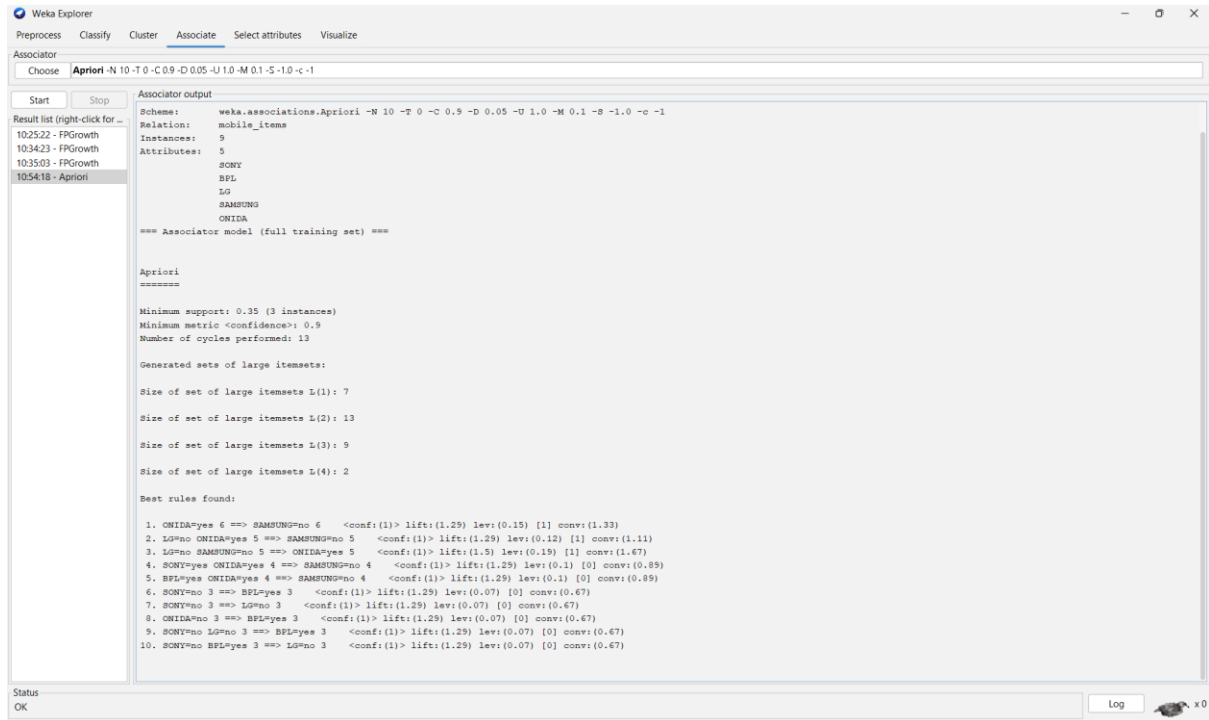
Status: OK

10. Create an ARFF file for the table below and implement for the Apriori Algorithm and FP growth algorithm and compare the rules generated by both the algorithms. Identify the unique rules generated by the above algorithms.

NOTE: Assume Min\_sup=2 and confidence= 50%

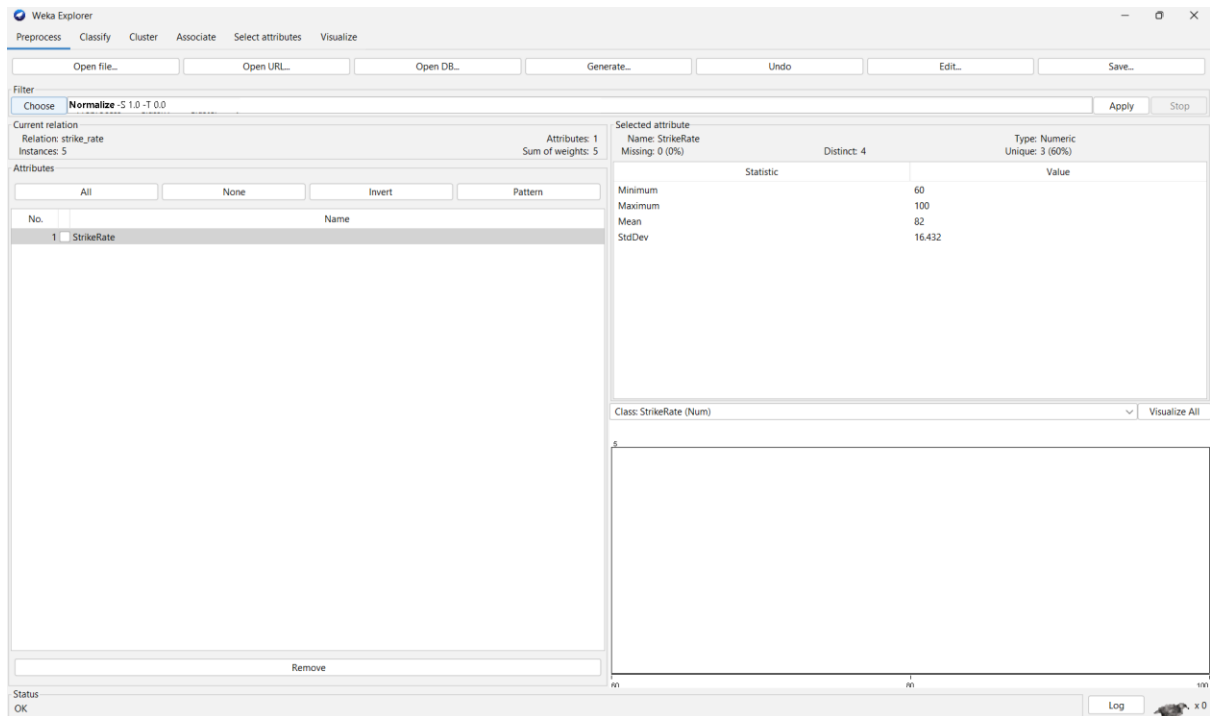
| T.ID | ITEMS              |
|------|--------------------|
| T1   | SONY, BPL, LG      |
| T2   | BPL, SAMSUNG       |
| T3   | BPL, ONIDA         |
| T4   | SONY, BPL, SAMSUNG |
| T5   | SONY, ONIDA        |

|    |                      |
|----|----------------------|
| T6 | BPL, ONIDA           |
| T7 | SONY, ONIDA          |
| T8 | SONY, BPL, ONIDA, LG |
| T9 | SONY, BPL, ONIDA     |



11., The given are the strike-rates scored by a batsman in season 1 in different tournaments.  
100, 70, 60, 90, 90

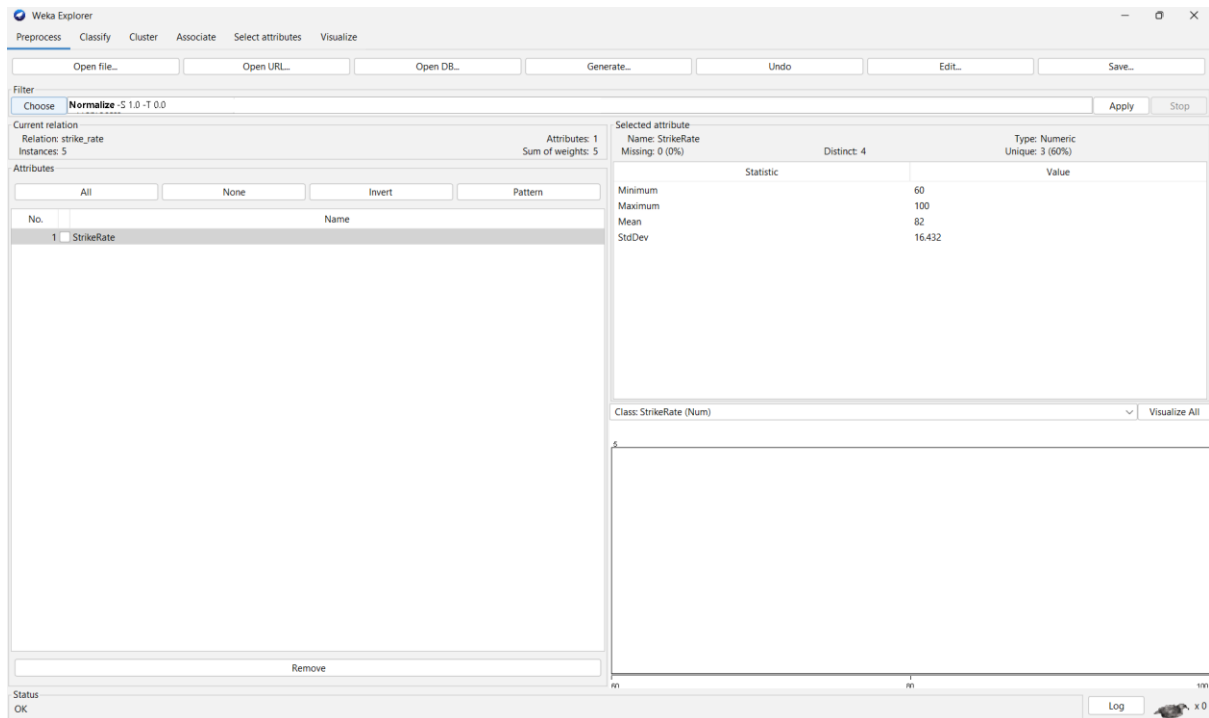
- min-max normalization by setting min = 0 and max = 1
- z-score normalization
- z-score normalization using the mean absolute deviation instead of standard deviation
- normalization by decimal scaling



12 Suppose some car is tested for the AvgSpeed and TotalTime data for 9 randomly selected car with the following result

|                        |    |    |    |    |    |    |    |    |    |
|------------------------|----|----|----|----|----|----|----|----|----|
| AvgSpeed<br>(in kph)   | 78 | 81 | 82 | 74 | 83 | 82 | 77 | 80 | 70 |
| TotalTime<br>(in mins) | 39 | 37 | 36 | 42 | 35 | 36 | 40 | 38 | 46 |

- Calculate the standard deviation of AvgSpeed and TotalTime.
- Calculate the Variance of AvgSpeed and TotalTime for the above dataset.



13. Consider this table

- c) **TID**    **items bought**
- d) **T100**   {M, O, N, K, E, Y}
- e) **T200**   {D, O, N, K, E, Y }
- f) **T300**   {M, A, K, E}
- g) **T400**   {M, U, C, K, Y}
- h) **T500**   {C, O, O, K, I, E}
- i) (a) Find all frequent item set using Apriori and FP-growth, respectively. Compare the efficiency of the two mining processes.
- j) (b) List all of the strong association rules (with support s and confidence c) matching the following metarule, where X is a variable representing customers, and item<sub>i</sub> denotes variables representing items (e.g., “A”, “B”, etc.):
- k)  $\forall x \in \text{transaction}, \text{buys}(X, \text{item1}) \wedge \text{buys}(X, \text{item2}) \Rightarrow \text{buys}(X, \text{item3})$

Weka Explorer

Preprocess    Classify    Cluster    **Associate**    Select attributes    Visualize

Associator

Choose    **Apriori** -N 10 -T 0 -C 0.9 -D 0.05 -U 1.0 -M 0.1 -S -1.0 -c -1

Start    Stop

Result list (right-click for ...)

- 10:25:22 - FPGrowth
- 10:34:23 - FPGrowth
- 10:35:03 - FPGrowth
- 10:54:18 - Apriori
- 11:03:24 - Apriori**

Associator output

```
M
O
N
K
E
Y
D
A
U
C
I

=== Associator model (full training set) ===

Apriori
=====

Minimum support: 0.85 (4 instances)
Minimum metric <confidence>: 0.9
Number of cycles performed: 3

Generated sets of large itemsets:

Size of set of large itemsets L(1): 6

Size of set of large itemsets L(2): 6

Size of set of large itemsets L(3): 1

Best rules found:

1. E=yes 4 ==> K=yes 4    <conf:(1)> lift:(1) lev:(0) [0] conv:(0)
2. D=no 4 ==> K=yes 4    <conf:(1)> lift:(1) lev:(0) [0] conv:(0)
3. A=no 4 ==> K=yes 4    <conf:(1)> lift:(1) lev:(0) [0] conv:(0)
4. U=no 4 ==> K=yes 4    <conf:(1)> lift:(1) lev:(0) [0] conv:(0)
5. I=no 4 ==> K=yes 4    <conf:(1)> lift:(1) lev:(0) [0] conv:(0)
6. U=no 4 ==> E=yes 4    <conf:(1)> lift:(1.25) lev:(0.16) [0] conv:(0.8)
7. E=yes 4 ==> U=no 4    <conf:(1)> lift:(1.25) lev:(0.16) [0] conv:(0.8)
8. E=yes U=no 4 ==> K=yes 4    <conf:(1)> lift:(1) lev:(0) [0] conv:(0)
9. K=yes U=no 4 ==> E=yes 4    <conf:(1)> lift:(1.25) lev:(0.16) [0] conv:(0.8)
10. K=yes E=yes 4 ==> U=no 4    <conf:(1)> lift:(1.25) lev:(0.16) [0] conv:(0.8)
```

Status